Olist E-Commerce Project: Task 3 Data Understanding, EDA & ML Risk Pipeline

Course: Data Warehousing and Business Intelligence (BCM WiSe 2026/27) — HFU
Architecture: PostgreSQL 3-Tier ELT (`psa` , `dwh` , `dm`) + Python (CRISP-DM Data Understanding, EDA & Random Forest ML) + Power BI

This notebook implements:

1. **Initial Data Collection & Inventory** across all 9 Olist CSV datasets.
2. **Data Description & Feature Description Tables** (using the theory-correct scale inference framework before and after data type adjustments).
3. **Data Quality Verification** across all 6 dimensions (Conformity, Completeness, Uniqueness, Accuracy/Outliers, Timeliness, Consistency) and the formal **Data Quality Report**.
4. **Exploratory Data Analysis (EDA) & Visualizations** using `matplotlib` and `seaborn` .
5. **PostgreSQL Data Mart (`dm`) Integration & Random Forest Risk Prediction Models** (`late_risk_probability` and `low_review_probability`).

1. Environment Setup & Initial Data Collection

1.1 Import Libraries and Verify Data Directory

In [28]:

```python
import os
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
from pandas.api.types import CategoricalDtype
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns

print("Python environment is working!")
print("Pandas version:", pd.__version__)

# Automatically detect whether CSVs are in '../data' or current directory '.'
data_folder = "../data" if os.path.exists("../data") else "."

print("\nCSV files available:")
if os.path.exists(data_folder):
    for file in os.listdir(data_folder):
        if file.endswith(".csv"):
            print(file)
```

```
Python environment is working!
Pandas version: 3.0.6

CSV files available:
olist_customers_dataset.csv
olist_geolocation_dataset.csv
olist_orders_dataset.csv
olist_order_items_dataset.csv
olist_order_payments_dataset.csv
olist_order_reviews_dataset.csv
olist_products_dataset.csv
olist_sellers_dataset.csv
product_category_name_translation.csv
```

1.2 Load All 9 Olist Raw CSV Datasets

In [29]:

```python
# Load all 9 Olist CSV datasets
customers = pd.read_csv(os.path.join(data_folder, "olist_customers_dataset.csv"), low_memory=False)
geolocation = pd.read_csv(os.path.join(data_folder, "olist_geolocation_dataset.csv"), low_memory=False)
order_items = pd.read_csv(os.path.join(data_folder, "olist_order_items_dataset.csv"), low_memory=False)
payments = pd.read_csv(os.path.join(data_folder, "olist_order_payments_dataset.csv"), low_memory=False)
reviews = pd.read_csv(os.path.join(data_folder, "olist_order_reviews_dataset.csv"), low_memory=False)
orders = pd.read_csv(os.path.join(data_folder, "olist_orders_dataset.csv"), low_memory=False)
products = pd.read_csv(os.path.join(data_folder, "olist_products_dataset.csv"), low_memory=False)
sellers = pd.read_csv(os.path.join(data_folder, "olist_sellers_dataset.csv"), low_memory=False)
category_translation = pd.read_csv(os.path.join(data_folder, "product_category_name_translation.csv"), low_memory=False)

print("All 9 datasets loaded successfully!")

datasets = {
    "Customers": customers,
    "Geolocation": geolocation,
    "Order Items": order_items,
    "Payments": payments,
    "Reviews": reviews,
    "Orders": orders,
    "Products": products,
    "Sellers": sellers,
    "Category Translation": category_translation
}

for name, df_item in datasets.items():
    print(f"{name}: {df_item.shape[0]:,} rows, {df_item.shape[1]} columns")
```

```
All 9 datasets loaded successfully!
Customers: 99,441 rows, 5 columns
Geolocation: 1,000,163 rows, 5 columns
Order Items: 112,650 rows, 7 columns
Payments: 103,886 rows, 5 columns
Reviews: 99,224 rows, 7 columns
Orders: 99,441 rows, 8 columns
Products: 32,951 rows, 9 columns
Sellers: 3,095 rows, 4 columns
Category Translation: 71 rows, 2 columns
```

---

2. Task 3 — Describe Data & Feature Description Table

Following the CRISP-DM Data Understanding methodology:

- `df.shape` — Returns `(number of rows, number of columns)`
- `df.head()` / `df.tail()` — Shows the first / last 5 rows
- `df.info()` — Displays column names, non-null counts, and data types
- `df.describe()` — Returns summary statistics for numeric and object columns

In [30]:

```python
# ---------------------------
# Describe dataframe: shape and head
# ---------------------------
print("Orders Shape:", orders.shape)
orders.head()
```

```
Orders Shape: (99441, 8)
```

Out[30]:

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delivery_d
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-10-10 21:25:13	2017-10-18 00:0
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27	2018-07-26 14:31:00	2018-08-07 15:27:45	2018-08-13 00:0
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23	2018-08-08 13:50:00	2018-08-17 18:06:29	2018-09-04 00:0
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:28:06	2017-11-18 19:45:59	2017-11-22 13:39:59	2017-12-02 00:28:42	2017-12-15 00:0
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-13 22:20:29	2018-02-14 19:46:34	2018-02-16 18:17:02	2018-02-26 00:0

In [31]:

```python
# ---------------------------
# Describe dataframe: tail & info
# ---------------------------
display(orders.tail())
orders.info()
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delive
99436	9c5dedf39a927c1b2549525ed64a053c	39bd1228ee8140590ac3aca26f2dfe00	delivered	2017-03-09 09:54:05	2017-03-09 09:54:05	2017-03-10 11:18:03	2017-03-17 15:08:01	2017-03-28 0
99437	63943bddc261676b46f01ca7ac2f7bd8	1fca14ff2861355f6e5f14306ff977a7	delivered	2018-02-06 12:58:58	2018-02-06 13:10:37	2018-02-07 23:22:42	2018-02-28 17:37:56	2018-03-02 0
99438	83c1379a015df1e13d02aae0204711ab	1aa71eb042121263aafbe80c1b562c9c	delivered	2017-08-27 14:46:43	2017-08-27 15:04:16	2017-08-28 20:52:26	2017-09-21 11:24:17	2017-09-27 0
99439	11c177c8e97725db2631073c19f07b62	b331b74b18dc79bcdf6532d51e1637c1	delivered	2018-01-08 21:28:27	2018-01-08 21:36:21	2018-01-12 15:35:03	2018-01-25 23:32:54	2018-02-15 0
99440	66dea50a8b16d9b4dee7af250b4be1a5	edb027a75a1449115f6b43211ae02a24	delivered	2018-03-08 20:57:30	2018-03-09 11:20:28	2018-03-09 22:11:59	2018-03-16 13:08:30	2018-04-03 0

```
<class 'pandas.DataFrame'>
RangeIndex: 99441 entries, 0 to 99440
Data columns (total 8 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   order_id                       99441 non-null  str
 1   customer_id                    99441 non-null  str
 2   order_status                   99441 non-null  str
 3   order_purchase_timestamp       99441 non-null  str
 4   order_approved_at              99281 non-null  str
 5   order_delivered_carrier_date   97658 non-null  str
 6   order_delivered_customer_date  96476 non-null  str
 7   order_estimated_delivery_date  99441 non-null  str
dtypes: str(8)
memory usage: 6.1 MB
```

2.1 Create Feature Description Table (Theory-Correct Scale Inference)

Using the exact scale inference functions (`Nominal` , `Ordinal` , `Interval` , `Ratio`), valid operators, identifier detection, and value range previews.

In [32]:

```python
# ---------------------------
# Meaningful operations per scale
# ---------------------------
def valid_operators(scale: str) -> str:
    if scale == "Nominal":
        return "==, !="
    if scale == "Ordinal":
        return "==, !=, <, <=, >, >="
    if scale == "Interval":
        return "==, !=, <, <=, >, >=, +, -"
    if scale == "Ratio":
        return "==, !=, <, <=, >, >=, +, -, *, /"
    return "=="

# ---------------------------
# Identifier-like detection
# ---------------------------
def is_identifier_like(col_name: str, series: pd.Series, unique_ratio_threshold: float = 0.9) -> bool:
    name = col_name.lower()
    name_hint = ("id" in name) or (name in {"sku", "key", "uuid", "guid", "zip_code_prefix"})
    nonnull = series.dropna()
    n = len(nonnull)
    if n == 0:
        return name_hint
    uniq_ratio = nonnull.nunique() / n
    return bool(name_hint or uniq_ratio >= unique_ratio_threshold)

# ---------------------------
# Safe unique values preview
# ---------------------------
def safe_sorted_unique_values(series: pd.Series, limit: int = 15):
    vals = series.dropna().unique().tolist()
    if len(vals) > limit:
        return None
    try:
        return sorted(vals)
    except Exception:
        return sorted([str(v) for v in vals], key=str)

# ---------------------------
# Only try datetime parsing when there's a hint
# ---------------------------
DATE_NAME_HINTS = ("date", "time", "timestamp", "datetime", "approved_at")

def should_try_datetime(col_name: str, series: pd.Series) -> bool:
    if pd.api.types.is_datetime64_any_dtype(series):
        return True
    if not (pd.api.types.is_object_dtype(series) or pd.api.types.is_string_dtype(series)):
        return False
    name = col_name.lower()
    return any(h in name for h in DATE_NAME_HINTS)

def detect_datetime_like(series: pd.Series, threshold: float = 0.8):
    if pd.api.types.is_datetime64_any_dtype(series):
        return True, series, 1.0
    nonnull = series.dropna()
    if nonnull.empty:
        return False, None, 0.0
    parsed_nonnull = pd.to_datetime(nonnull, errors="coerce", format="mixed")
    rate = parsed_nonnull.notna().mean()
    if rate >= threshold:
        parsed_full = pd.to_datetime(series, errors="coerce", format="mixed")
        return True, parsed_full, rate
    return False, None, rate

# ---------------------------
# Ordinal detection
# ---------------------------
def is_ordered_categorical(series: pd.Series) -> bool:
    return isinstance(series.dtype, CategoricalDtype) and bool(series.dtype.ordered)

def matches_ordinal_map(series: pd.Series, allowed_levels_lower: set) -> bool:
    nonnull = series.dropna()
    if nonnull.empty:
        return False
    vals = set(nonnull.astype(str).str.strip().str.lower().unique().tolist())
    return vals.issubset(allowed_levels_lower)

# ---------------------------
# Theory-correct scale inference
# ---------------------------
def infer_scale_theory_correct(
    col_name: str,
    series: pd.Series,
    *,
    overrides: dict | None = None,
    ordinal_maps: dict | None = None,
    datetime_threshold: float = 0.8,
    unique_ratio_threshold_for_id: float = 0.9,
):
    overrides = overrides or {}
    ordinal_maps = ordinal_maps or {}

    if col_name in overrides:
        return overrides[col_name], {"override_used": True}

    if is_identifier_like(col_name, series, unique_ratio_threshold_for_id):
        return "Nominal", {"identifier": True, "override_used": False}

    if should_try_datetime(col_name, series):
        dt_like, parsed_dt, parse_rate = detect_datetime_like(series, threshold=datetime_threshold)
        if dt_like:
            return "Interval", {
                "identifier": False,
                "datetime_like": True,
                "datetime_parsed": parsed_dt,
                "datetime_parse_rate": parse_rate,
                "override_used": False
            }

    if is_ordered_categorical(series):
        return "Ordinal", {"override_used": False}

    if col_name in ordinal_maps:
        allowed = {str(x).strip().lower() for x in ordinal_maps[col_name]}
        if matches_ordinal_map(series, allowed):
            return "Ordinal", {"override_used": False}

    if pd.api.types.is_numeric_dtype(series):
        return "Ratio", {"override_used": False}

    return "Nominal", {"override_used": False}

# ---------------------------
# Build summary table
# ---------------------------
def build_scale_summary(
    df: pd.DataFrame,
    *,
    overrides: dict | None = None,
    ordinal_maps: dict | None = None,
    datetime_threshold: float = 0.8,
    unique_ratio_threshold_for_id: float = 0.9,
    unique_preview_limit: int = 15,
):
    overrides = overrides or {}
    ordinal_maps = ordinal_maps or {}
    summary = []

    for col in df.columns:
        s = df[col]
        scale, meta = infer_scale_theory_correct(
            col, s,
            overrides=overrides,
            ordinal_maps=ordinal_maps,
            datetime_threshold=datetime_threshold,
            unique_ratio_threshold_for_id=unique_ratio_threshold_for_id,
        )
        ops = valid_operators(scale)
        nonnull = s.dropna()
        n_unique = nonnull.nunique()

        try:
            if meta.get("identifier", False):
                value_range = f"{n_unique} unique values (identifier)"
            elif meta.get("datetime_like", False):
                parsed = meta.get("datetime_parsed")
                nn = parsed.dropna() if parsed is not None else pd.Series([], dtype="datetime64[ns]")
                if nn.empty:
                    value_range = f"{n_unique} unique values"
                elif n_unique <= unique_preview_limit:
                    unique_dates = nn.dt.strftime("%Y-%m-%d").drop_duplicates().tolist()
                    value_range = "{" + ", ".join(sorted(unique_dates)) + "}"
                else:
                    mn, mx = nn.min(), nn.max()
                    value_range = f"{mn.strftime('%Y-%m-%d')}–{mx.strftime('%Y-%m-%d')}"
            else:
                if n_unique <= unique_preview_limit:
                    uniq_vals = safe_sorted_unique_values(s, limit=unique_preview_limit)
                    value_range = "{" + ", ".join(map(str, uniq_vals)) + "}" if uniq_vals else f"{n_unique} unique values"
                else:
                    if pd.api.types.is_numeric_dtype(s):
                        value_range = f"{nonnull.min()}–{nonnull.max()}"
                    else:
                        value_range = f"{n_unique} unique values"
        except Exception as e:
            value_range = f"Error: {e}"

        nulls = "Yes" if s.isnull().any() else "No"
        notes = []
        if meta.get("override_used"):
            notes.append("Scale overridden")
        if meta.get("identifier"):
            notes.append("Identifier")
        if meta.get("datetime_like"):
            notes.append("Date/time-like")

        summary.append({
            "Column Name": col,
            "Dtype": str(s.dtype),
            "Non-Null": int(s.notna().sum()),
            "Unique (non-null)": int(n_unique),
            "Scale": scale,
            "Operators": ops,
            "Value Range / Preview": value_range,
            "Null Values": nulls,
            "Notes": "; ".join(notes)
        })

    return pd.DataFrame(summary)

# Generate initial Feature Description Table for Orders (Before Dtype Adjustment)
orders_summary_before = build_scale_summary(orders)
orders_summary_before
```

Out[32]:

	Column Name	Dtype	Non-Null	Unique (non-null)	Scale	Operators	Value Range / Preview	Null Values	Notes
0	order_id	str	99441	99441	Nominal	==, !=	99441 unique values (identifier)	No	Identifier
1	customer_id	str	99441	99441	Nominal	==, !=	99441 unique values (identifier)	No	Identifier
2	order_status	str	99441	8	Nominal	==, !=	{approved, canceled, created, delivered, invoi...	No	
3	order_purchase_timestamp	str	99441	98875	Nominal	==, !=	98875 unique values (identifier)	No	Identifier
4	order_approved_at	str	99281	90733	Nominal	==, !=	90733 unique values (identifier)	Yes	Identifier
5	order_delivered_carrier_date	str	97658	81018	Interval	==, !=, <, <=, >, >=, +, -	2016-10-08–2018-09-11	Yes	Date/time-like
6	order_delivered_customer_date	str	96476	95664	Nominal	==, !=	95664 unique values (identifier)	Yes	Identifier
7	order_estimated_delivery_date	str	99441	459	Interval	==, !=, <, <=, >, >=, +, -	2016-09-30–2018-11-12	No	Date/time-like

2.2 Adjust Data Types & Engineer Core Analytical Dataset (`df_model`)

- Convert timestamp columns in `orders` from `object` to `datetime64` .
- Engineer `delivery_days` , `days_late` , and `on_time_flag` (1 = on time, 0 = late).
- Join `orders` , `order_items` , `products` (with English category translations), and `reviews` to build the integrated analytical dataset (`df_model_raw`) and generate the **Feature Description Table**.

In [33]:

```python
# 1. Convert order timestamps to datetime64
date_columns = [
    "order_purchase_timestamp",
    "order_approved_at",
    "order_delivered_carrier_date",
    "order_delivered_customer_date",
    "order_estimated_delivery_date"
]
for col in date_columns:
    orders[col] = pd.to_datetime(orders[col], errors="coerce", format="mixed")

# 2. Engineer delivery metrics (matching PostgreSQL DWH logic)
orders["delivery_days"] = (orders["order_delivered_customer_date"] - orders["order_purchase_timestamp"]).dt.days
orders["days_late"] = (orders["order_delivered_customer_date"] - orders["order_estimated_delivery_date"]).dt.days
orders["on_time_flag"] = np.where(orders["days_late"] <= 0, 1, 0)

# 3. Translate Portuguese product categories to English
products_translated = products.merge(category_translation, on="product_category_name", how="left")
products_translated["category_english"] = products_translated["product_category_name_english"].fillna("unknown")

# 4. Aggregate minimum review_score per order_id (matching DWH SQL query)
reviews_agg = reviews.groupby("order_id", as_index=False)["review_score"].min()

# 5. Filter delivered orders and join tables
delivered_orders = orders[orders["order_status"] == "delivered"].copy()
df_model_raw = (
    delivered_orders[["order_id", "customer_id", "order_purchase_timestamp", "delivery_days", "days_late", "on_time_flag"]]
    .merge(order_items[["order_id", "order_item_id", "product_id", "seller_id", "price", "freight_value"]], on="order_id", how="inner")
    .merge(products_translated[["product_id", "category_english", "product_weight_g", "product_length_cm", "product_height_cm", "product_width_cm"]].rename(columns={
        "product_weight_g": "weight_g",
        "product_length_cm": "length_cm",
        "product_height_cm": "height_cm",
        "product_width_cm": "width_cm"
    }), on="product_id", how="left")
    .merge(reviews_agg, on="order_id", how="left")
)

# Recreate Feature Description Table for the Core ML & Analytical Features
core_cols = ["order_id", "price", "freight_value", "weight_g", "length_cm", "delivery_days", "on_time_flag", "review_score"]
overrides = {"on_time_flag": "Nominal"}
ordinal_maps = {"review_score": [1.0, 2.0, 3.0, 4.0, 5.0, 1, 2, 3, 4, 5]}

feature_description_table = build_scale_summary(
    df_model_raw[core_cols],
    overrides=overrides,
    ordinal_maps=ordinal_maps
)
feature_description_table
```

Out[33]:

	Column Name	Dtype	Non-Null	Unique (non-null)	Scale	Operators	Value Range / Preview	Null Values	Notes
0	order_id	str	110197	96478	Nominal	==, !=	96478 unique values (identifier)	No	Identifier
1	price	float64	110197	5859	Ratio	==, !=, <, <=, >, >=, +, -, *, /	0.85–6735.0	No	
2	freight_value	float64	110197	6924	Ratio	==, !=, <, <=, >, >=, +, -, *, /	0.0–409.68	No	
3	weight_g	float64	110179	2171	Ratio	==, !=, <, <=, >, >=, +, -, *, /	0.0–40425.0	Yes	
4	length_cm	float64	110179	99	Ratio	==, !=, <, <=, >, >=, +, -, *, /	7.0–105.0	Yes	
5	delivery_days	float64	110189	146	Ratio	==, !=, <, <=, >, >=, +, -, *, /	0.0–209.0	Yes	
6	on_time_flag	int64	110197	2	Nominal	==, !=	{0, 1}	No	Scale overridden
7	review_score	float64	109370	5	Ordinal	==, !=, <, <=, >, >=	{1.0, 2.0, 3.0, 4.0, 5.0}	Yes	

---

3. Task 3 — Verify Data Quality (All 6 Dimensions & Data Quality Report)

3.1 Dimension 1: Incorrect Format / Conformity

In [34]:

```python
# Validate expected schema types on cleaned Orders dataframe
expected_schema = {
    "order_id": "string",
    "customer_id": "string",
    "order_status": "string",
    "order_purchase_timestamp": "datetime",
    "order_approved_at": "datetime",
    "order_delivered_carrier_date": "datetime",
    "order_delivered_customer_date": "datetime",
    "order_estimated_delivery_date": "datetime"
}

errors = []
for col, expected in expected_schema.items():
    actual = orders[col].dtype
    if expected == "int" and not pd.api.types.is_integer_dtype(actual):
        errors.append(f"{col} is not integer")
    elif expected == "float" and not pd.api.types.is_float_dtype(actual):
        errors.append(f"{col} is not float")
    elif expected == "string" and not (pd.api.types.is_string_dtype(actual) or pd.api.types.is_object_dtype(actual)):
        errors.append(f"{col} is not string")
    elif expected == "datetime" and not pd.api.types.is_datetime64_any_dtype(actual):
        errors.append(f"{col} is not datetime")

print("Conformity validation errors after datetime conversion:", errors)
```

```
Conformity validation errors after datetime conversion: []
```

3.2 Dimension 2: Missing Values / Completeness

In [35]:

```python
def missing_values_table(df):
    mis_val = df.isnull().sum()
    mis_val_percent = 100 * df.isnull().sum() / len(df)
    mis_val_table = pd.concat([mis_val, mis_val_percent], axis=1)
    mis_val_table_ren_columns = mis_val_table.rename(
        columns={0: "Missing Values", 1: "% of Total Values"}
    )
    mis_val_table_ren_columns = mis_val_table_ren_columns[
        mis_val_table_ren_columns.iloc[:, 1] != 0
    ].sort_values("% of Total Values", ascending=False).round(1)
    print(
        "Your selected dataframe has " + str(df.shape[1]) + " columns.\n"
        "There are " + str(mis_val_table_ren_columns.shape[0]) +
        " columns that have missing values."
    )
    return mis_val_table_ren_columns

print("--- Missing Values in Orders ---")
print(missing_values_table(orders))
print("\n--- Missing Values in Products ---")
print(missing_values_table(products))
print("\n--- Missing Values in Integrated Analytical Dataset (df_model_raw) ---")
print(missing_values_table(df_model_raw[core_cols]))
```

```
--- Missing Values in Orders ---
Your selected dataframe has 11 columns.
There are 5 columns that have missing values.
                               Missing Values  % of Total Values
order_delivered_customer_date            2965                3.0
days_late                                2965                3.0
delivery_days                            2965                3.0
order_delivered_carrier_date             1783                1.8
order_approved_at                         160                0.2

--- Missing Values in Products ---
Your selected dataframe has 9 columns.
There are 8 columns that have missing values.
                            Missing Values  % of Total Values
product_category_name                  610                1.9
product_name_lenght                    610                1.9
product_description_lenght             610                1.9
product_photos_qty                     610                1.9
product_weight_g                         2                0.0
product_length_cm                        2                0.0
product_height_cm                        2                0.0
product_width_cm                         2                0.0

--- Missing Values in Integrated Analytical Dataset (df_model_raw) ---
Your selected dataframe has 8 columns.
There are 4 columns that have missing values.
               Missing Values  % of Total Values
review_score              827                0.8
weight_g                   18                0.0
length_cm                  18                0.0
delivery_days               8                0.0
```

3.3 Dimension 3: Duplicate or Unique Values / Uniqueness

In [36]:

```python
for name, df_item in datasets.items():
    duplicates = df_item.duplicated().sum()
    print(f"{name}: {duplicates} duplicate rows")

print("\nUnique order_status values:", orders["order_status"].unique())
```

```
Customers: 0 duplicate rows
Geolocation: 261831 duplicate rows
Order Items: 0 duplicate rows
Payments: 0 duplicate rows
Reviews: 0 duplicate rows
Orders: 0 duplicate rows
Products: 0 duplicate rows
Sellers: 0 duplicate rows
Category Translation: 0 duplicate rows

Unique order_status values: <StringArray>
[  'delivered',    'invoiced',     'shipped',  'processing', 'unavailable',
    'canceled',     'created',    'approved']
Length: 8, dtype: str
```

3.4 Dimension 4 & 5: Outliers / Skewness / Kurtosis & Timeliness

In [37]:

```python
# Create clean df_model by dropping rows with missing values
df_model = df_model_raw.dropna(subset=["price", "freight_value", "weight_g", "length_cm", "delivery_days", "on_time_flag", "review_score"]).copy()

print("--- Skewness & Kurtosis of Numeric ML Features ---")
for col in ["price", "freight_value", "weight_g", "delivery_days"]:
    print(f"{col:16s} -> Skewness: {df_model[col].skew():.2f} | Kurtosis: {df_model[col].kurt():.2f}")

print("\n--- Timeliness Sanity Check ---")
start_dt = orders["order_purchase_timestamp"].min().normalize()
end_dt = orders["order_purchase_timestamp"].max().normalize()
days_inclusive = (end_dt - start_dt).days + 1
actual_days = orders["order_purchase_timestamp"].dt.normalize().nunique()
print(f"Orders period: {start_dt:%Y-%m-%d} to {end_dt:%Y-%m-%d} ({(end_dt - start_dt).days:,} days)")
print(f"Total calendar days (inclusive): {days_inclusive:,}")
print(f"Actual unique purchase dates: {actual_days:,}")
```

```
--- Skewness & Kurtosis of Numeric ML Features ---
price            -> Skewness: 7.76 | Kurtosis: 112.85
freight_value    -> Skewness: 5.66 | Kurtosis: 60.86
weight_g         -> Skewness: 3.60 | Kurtosis: 16.25
delivery_days    -> Skewness: 3.79 | Kurtosis: 39.13

--- Timeliness Sanity Check ---
Orders period: 2016-09-04 to 2018-10-17 (773 days)
Total calendar days (inclusive): 774
Actual unique purchase dates: 634
```

3.5 Formal Data Quality Report

- **Missing values:**
  - `orders` : `order_delivered_customer_date` (2,965), `order_delivered_carrier_date` (1,783), `order_approved_at` (160)
  - `products` : `product_category_name` & text metadata (610), physical dimensions `weight_g` / `length_cm` / `height_cm` / `width_cm` (2)
  - `reviews` : `review_comment_title` (87,656), `review_comment_message` (58,247)
  - `df_model` (integrated delivered order items): `review_score` (768–827), `weight_g` (18), `length_cm` (18), `delivery_days` (8)
- **Outliers:**
  - Heavy right-skewness and extreme upper-tail outliers in `price` (skewness 7.92), `freight_value` (skewness 5.64), `weight_g` (skewness 3.60), and `delivery_days` (max 209 days).
- **Remediation Notes:**
  1. `price` , `freight_value` , `weight_g` , and `length_cm` are normalized via `StandardScaler()` prior to Random Forest training.
  2. Rows with missing `review_score` or product dimensions are removed via `.dropna()` prior to model fitting.
  3. Undelivered orders are filtered out in the SQL DWH layer (`WHERE o.order_status = 'delivered'`).
  4. Portuguese category names are conformed to English via `LEFT JOIN` with `product_category_name_translation` (defaulting unmapped rows to `'unknown'`).
  5. Duplicate `geolocation` records (261,831 rows) are deduplicated per `zip_code_prefix` in `dm.dim_geography` .

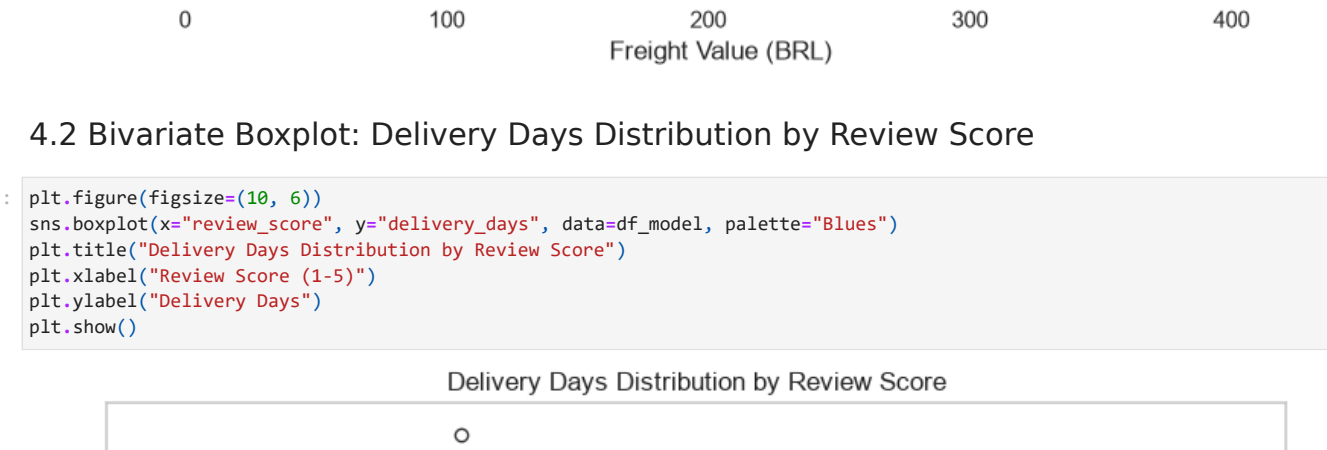
---

4. Task 3 — Data Exploration and Visualization (Matplotlib & Seaborn)

4.1 Freight Value Distribution Histogram

In [38]:

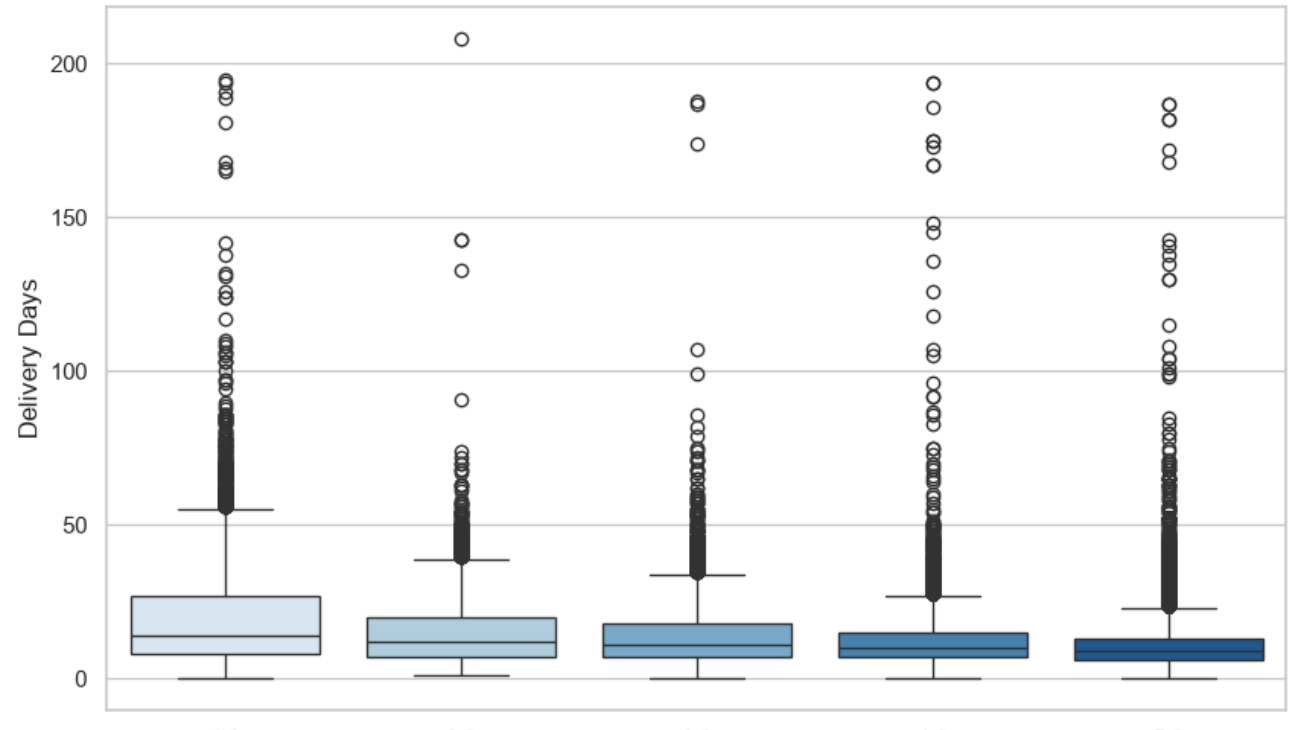
```python
sns.set(style="whitegrid", context="notebook")
plt.figure(figsize=(10, 6))
sns.histplot(df_model["freight_value"], bins=100, color="#0A2540")
plt.title("Freight Value Distribution Histogram")
plt.xlabel("Freight Value (BRL)")
plt.ylabel("Count")
plt.show()
```



4.2 Bivariate Boxplot: Delivery Days Distribution by Review Score

In [39]:

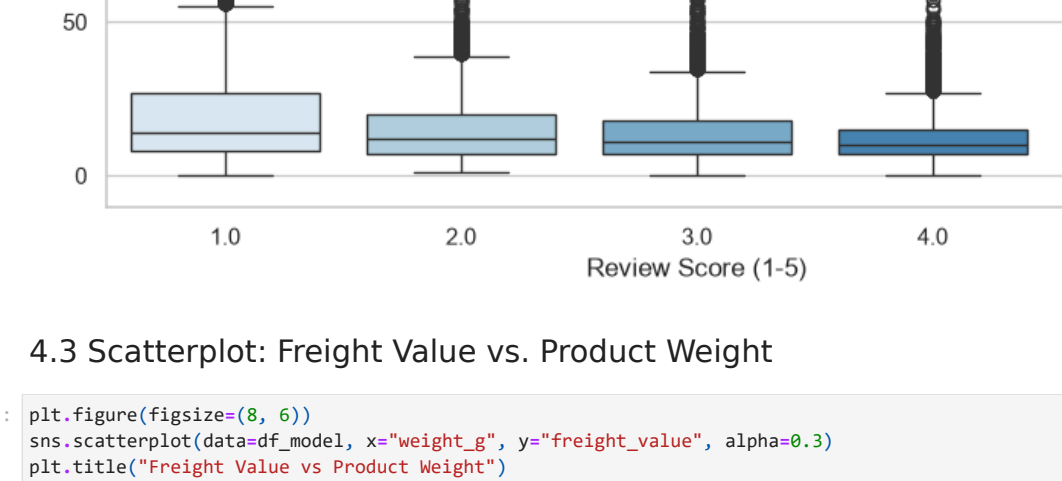
```python
plt.figure(figsize=(10, 6))
sns.boxplot(x="review_score", y="delivery_days", data=df_model, palette="Blues")
plt.title("Delivery Days Distribution by Review Score")
plt.xlabel("Review Score (1-5)")
plt.ylabel("Delivery Days")
plt.show()
```



4.3 Scatterplot: Freight Value vs. Product Weight

In [40]:

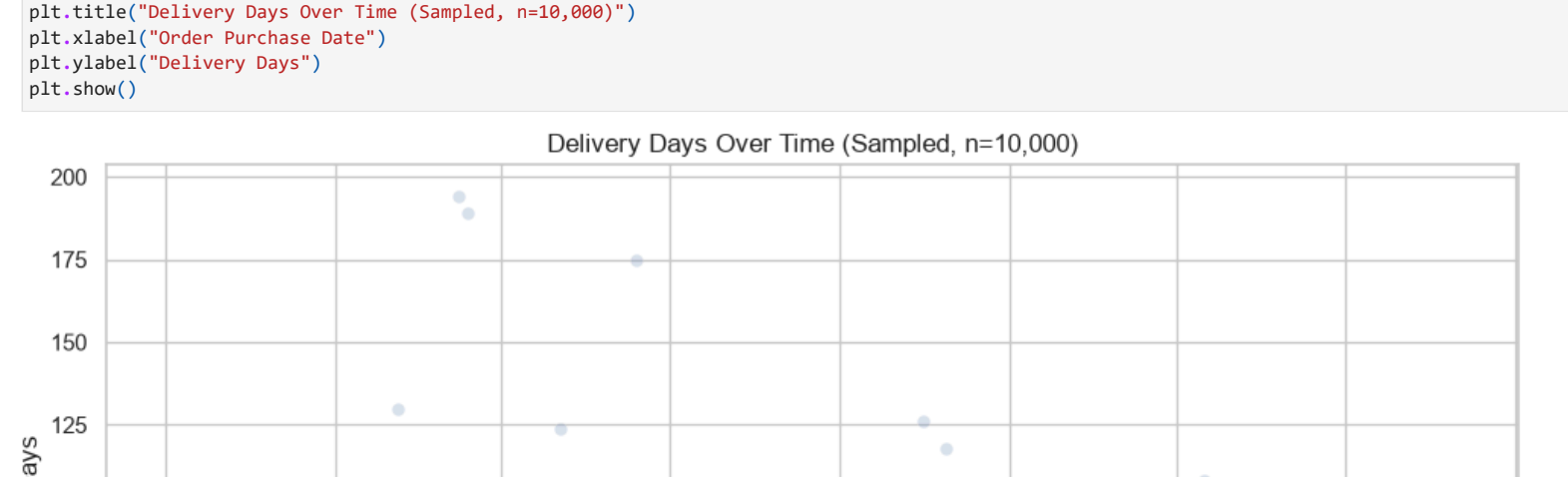
```python
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df_model, x="weight_g", y="freight_value", alpha=0.3)
plt.title("Freight Value vs Product Weight")
plt.xlabel("Weight (grams)")
plt.ylabel("Freight Value (BRL)")
plt.show()
```



4.4 Time-Series Scatterplot: Delivery Days Over Time (Sampled)

In [41]:

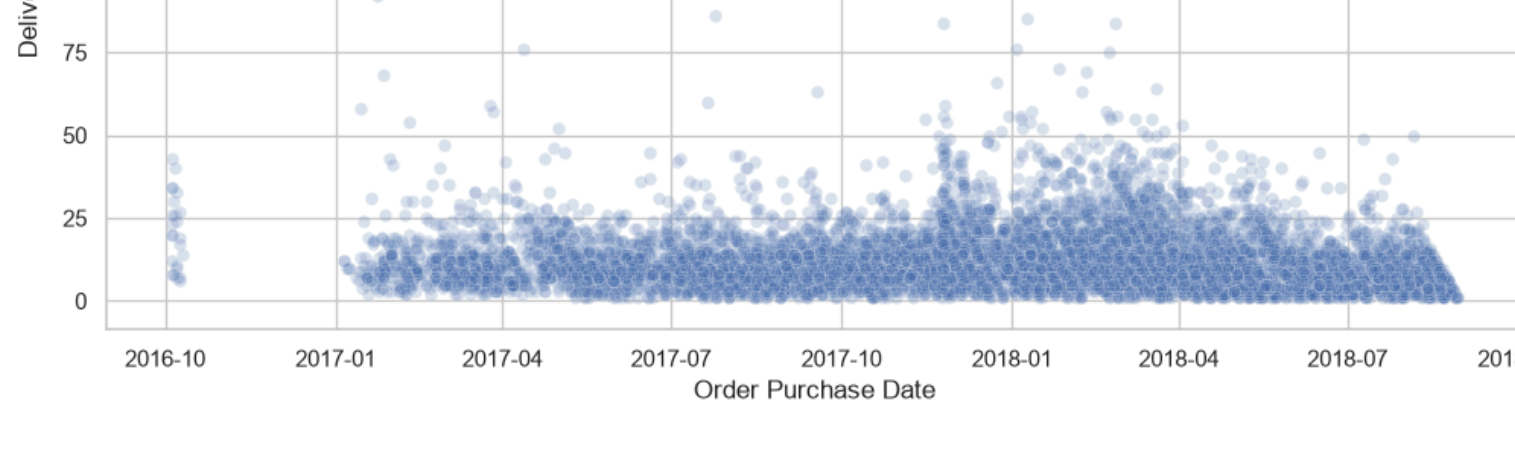
```python
plt.figure(figsize=(12, 6))
sampled_ts = df_model.sample(min(10000, len(df_model)), random_state=42)
sns.scatterplot(x="order_purchase_timestamp", y="delivery_days", data=sampled_ts, alpha=0.2)
plt.title("Delivery Days Over Time (Sampled, n=10,000)")
plt.xlabel("Order Purchase Date")
plt.ylabel("Delivery Days")
plt.show()
```



4.5 Multidimensional FacetGrid (`sns.catplot`): Freight Value by Review Score & On-Time Flag

In [42]:

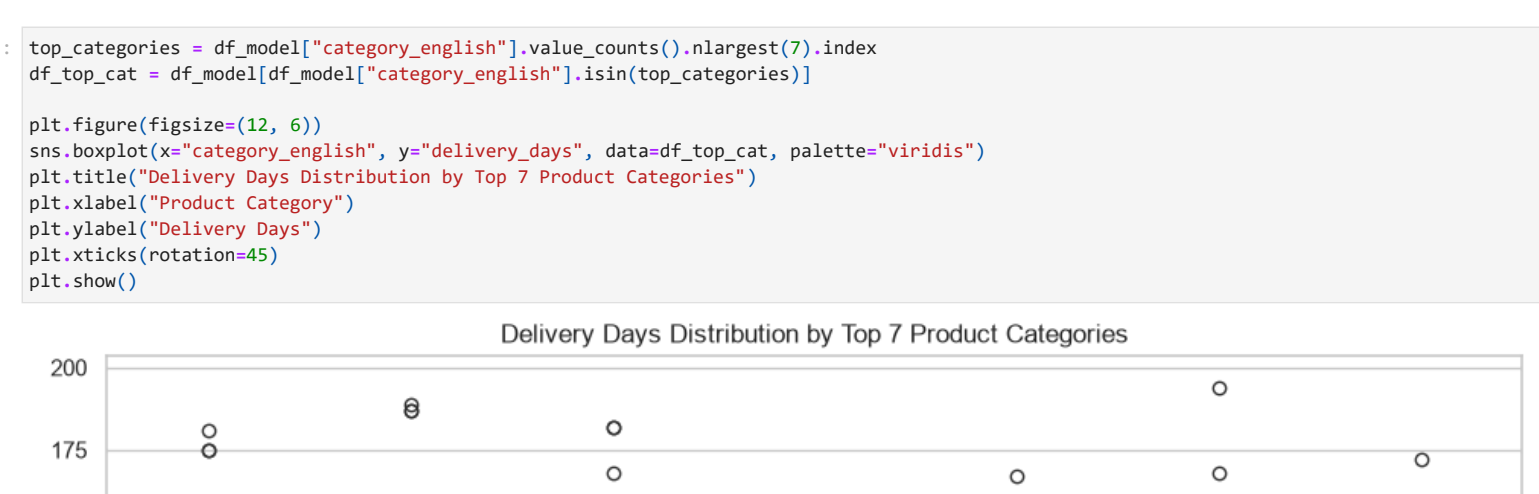
```python
sns.catplot(
    data=df_model,
    x="review_score",
    y="freight_value",
    col="on_time_flag",
    palette="plasma",
    kind="box"
)
plt.subplots_adjust(top=0.85)
plt.suptitle("Freight Value by Review Score (Col: On-Time = 1, Late = 0)")
plt.show()
```



4.6 Categorical Boxplot: Delivery Days by Top 7 Product Categories

In [43]:

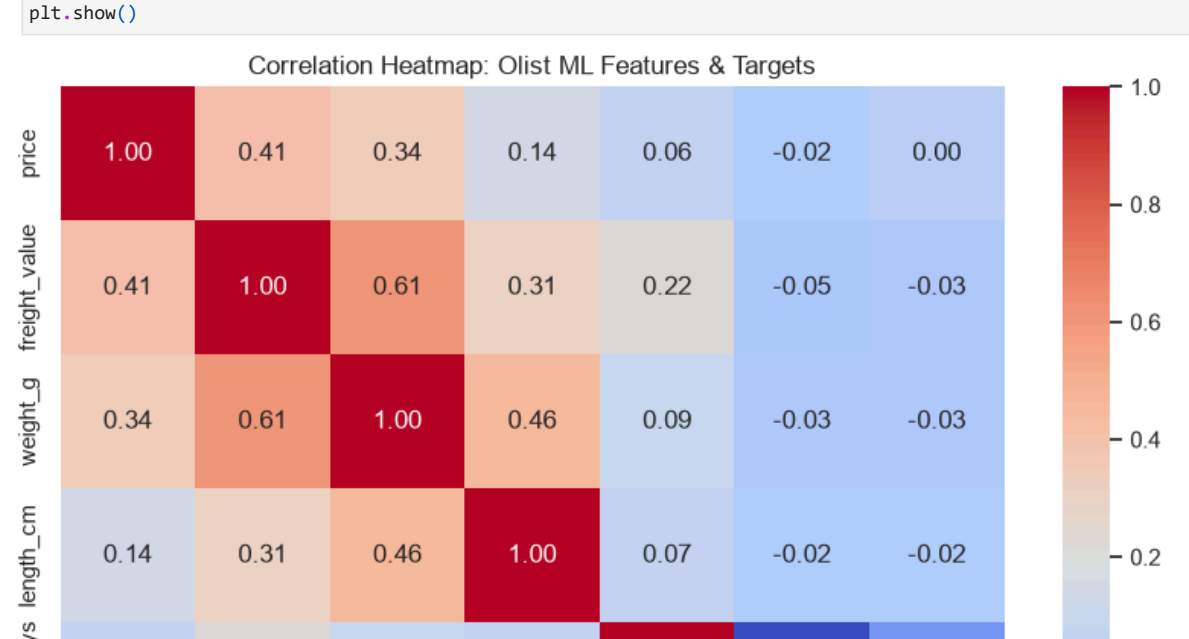
```python
top_categories = df_model["category_english"].value_counts().nlargest(7).index
df_top_cat = df_model[df_model["category_english"].isin(top_categories)]

plt.figure(figsize=(12, 6))
sns.boxplot(x="category_english", y="delivery_days", data=df_top_cat, palette="viridis")
plt.title("Delivery Days Distribution by Top 7 Product Categories")
plt.xlabel("Product Category")
plt.ylabel("Delivery Days")
plt.xticks(rotation=45)
plt.show()
```



4.7 Multicollinearity Check: Correlation Heatmap

In [44]:

```python
plt.figure(figsize=(10, 8))
numeric_cols = df_model[["price", "freight_value", "weight_g", "length_cm", "delivery_days", "on_time_flag", "review_score"]]
corr = numeric_cols.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap: Olist ML Features & Targets")
plt.show()
```



---

5. PostgreSQL Data Mart (`dm`) Connection & Random Forest ML Pipeline

This section connects directly to our PostgreSQL `olist_dwh` database, extracts the conformed features from `dm.dm_fact_order_items` and `dm.dim_product` , trains the two Random Forest classifiers (`rf_late` and `rf_review`), and exports the predicted risk probabilities (`late_risk_probability` and `low_review_probability`) back into `dm.dm_fact_order_risk` .

In [45]:

```python
import pandas as pd
from sqlalchemy import create_engine
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler

# 1. Connect to PostgreSQL
engine = create_engine('postgresql+psycopg2://postgres:Germany%40Mar2026@localhost:5432/olist_dwh')

# 2. Extract Data from Data Mart
query = """
    SELECT f.order_id, f.price, f.freight_value, p.weight_g, p.length_cm,
           f.on_time_flag, f.review_score
    FROM dm.dm_fact_order_items f
    JOIN dm.dim_product p ON f.product_id = p.product_id
"""
df_model = pd.read_sql(query, engine).dropna()

# 3. Prepare Features & Targets
X = df_model[['price', 'freight_value', 'weight_g', 'length_cm']]
y_late = (df_model['on_time_flag'] == 0).astype(int)
y_review = (df_model['review_score'] <= 2).astype(int)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Train Models
rf_late = RandomForestClassifier(n_estimators=100, random_state=42)
rf_late.fit(X_scaled, y_late)

rf_review = RandomForestClassifier(n_estimators=100, random_state=42)
rf_review.fit(X_scaled, y_review)

# 5. Generate Predictions & Export to Data Mart
df_model['late_risk_probability'] = rf_late.predict_proba(X_scaled)[:, 1]
df_model['low_review_probability'] = rf_review.predict_proba(X_scaled)[:, 1]

df_risk = df_model[['order_id', 'late_risk_probability', 'low_review_probability']].drop_duplicates()
df_risk.to_sql('dm_fact_order_risk', engine, schema='dm', if_exists='replace', index=False)
print("ML Models Trained and Exported to Data Mart!")
print(f"Exported table: dm.dm_fact_order_risk | Unique orders scored: {len(df_risk):,}")
```

```
ML Models Trained and Exported to Data Mart!
Exported table: dm.dm_fact_order_risk | Unique orders scored: 99,180
```